

-
- A. Define typical accessor/getter function for a data member named **center** of type **Point** in a class **Circle**.
- B. Define typical mutator/setter function for a data member of **Set** class named **size** of type **long**. Setter function must implement a check on **size** as non-negative.
- C. Write definition of a destructor of **MSWord** class to close the file pointed by a *data member* (ifstream objects) named *inFile*.
- D. Write two different prototypes of the Copy constructor for a class named **Cashmemo** whose members are *serial_number*, *date*, *customer* and *sale_lines*.
- E. Write the definition of the explicit operator **double** for class named **RationalNumber** composed two integers *num* and *den*. An example rational number is at right whose **double** equivalent is *num/den*.
- F. Write definition for the overloading operator **<=** as a member **ComplexNumber** class. Two complex numbers have to be compared through their modulus ($\sqrt{x^2 + y^2}$).

- G. Class Point has only one constructor with two parameters. Class Polygon has to store N (variable) number of point class objects. write Polygon class with appropriate data members and constructor with one parameter (N). Call point class constructor with random values?

- H. Complete following code to save object **obj** in both files.

```
int main() {  
    ofstream out1("data.txt"); //text file  
    ofstream out2("data.bin", ios::binary);  
    Simple obj(...);  
}
```

```
class Simple{  
public:  
    int x;  
    float f;  
    double d;  
};
```

- I. If member function perfectly **copy** (making deep copy) for class **Klacc** is available, write the definition of **copy constructor** & **assignment operator** using **copy** function?

```
class Klacc{  
    //Use for dynamic arrays  
    int *c; // for 1d  
    float **d; // for 2d  
};
```

- J. Give an example with necessary code of any constructor of class named **AAA**, to initialize its *const data member* **CC** of type **double** with the value provided as a parameter.

Write code/functions for the following descriptions.

- A. You are required to design a **class Circle** with the data members **int cx**, **int cy**, **double radius** and **static double pi** where **cx** and **cy** are co-ordinates of the center point and **pi** is the variable initialized to **3.14** required to compute area of a circle using formula πr^2 . You are required to define the **Circle** at least, that allow following **main()** code to execute correctly. The output of code is also written to show the effect of each code statement. From code in **main()** and the output provided **identify** the **functions** and **operators** that you must **overload** and write them down neatly with correct prototypes and implementation.

```
int main (){
    Circle c1;                //initializes x,y and radius to 0
    Circle c2(2,3,7.5);      //inits x,y & radius to 2,3 & 7.5 respectively
    cout << c1 << c2;        //print as shown below
    Circle c3 = c2++;         //radius of c2 is post incremented by 1
    cout << c2 << c3;
    c1.set(4,8,21.30);        //updates c1 using operator()( . . . . )
    cout << c1;
    if(c1>=c2){               //A circle is greater if its area is greater
        cout << "Job done!" << endl;
    }
}
```

Output:

```
Circle at (0,0) has radius 0
Circle at (2,3) has radius 7.5
Circle at (2,3) has radius 8.5
Circle at (2,3) has radius 7.5
Circle at (4,8) has radius 21.3
Job done!
```


- B. Write class **Job** having data member: **time duration** of type **double** and **count of workers** as **integer**. The valid range of **time duration** is 1 to 1000 with default value 8 and the valid range of **count of workers** is above 4 with default value 10. An example functions is given below to elaborate the Job class further. You have to create the next functions related to Job class.

```
istream& operator>>(istream &in, Job &job){
```

```
    double d;    int c;
    in >> d >> c;
    if(d<1 || d>14 || c < 3)
        cout<<"invalid job data";
    job.setDuration(d);
    job.setWorkers(c);
    return in;
}
```

i) Setter for duration

iii) Pre decrement operator — overload as member function, which reduces the count of workers by 10 along with perform everything related to operator —.

ii) Multiplication operator overload as global function to multiply a Job and an integer results in a Job with duration multiplied with the integer. If duration is crossing the limit, it has to be set to its maximum.

iv) Main logic (main function) to read jobs from a text files named jobs.txt having count of jobs say N at line 1 and then N lines each with a job data. After reading data from file, decrement each job, display each job and save updated job to file again. You must assume that all appropriate function and operators are available in Job class.

- c. Give output of main programs given at end of this question, each using the Movie class defined as follows.

```
class Movie
{
public:
    Movie() { }
    Movie(Movie& m)    {
        *this = m;
        cout << "Movie::Copy ctor" << endl;
    }
    Movie play() {
        cout << "Movie::Play()" << endl;
        return *this;
    }
    Movie rewind()    {
        cout << "Movie::Rewind()" << endl;
        return *this;
    }
};
//global function
void rateMovie(Movie m){
    cout << "Movie rating complete." << endl;
}
```

Part A:

<pre>int main(){ Movie m; m.play().rewind(); }</pre>	Output:
--	---------

Part B:

<pre>int main(){ Movie m; rateMovie(m.play()); }</pre>	Output:
--	---------

- A. Consider the following code as error free and answer two question asked later and at end.

```
class Integer{
    long id;
    int value;
public:
    static long objN;
    long & getId() const{
        return this->id;
    }
    void setId(long id) {
        this->id = id;
    }
    int& getValue() const{
        return this->value;
    }
    void setValue(int value) {
        this->value = value;
    }
    Integer(){
        this->setId(++Integer::objN);
    }
    Integer(int value) {
        this->setId(++Integer::objN);
        this->setValue(value);
    }
    Integer(const Integer &integer) {
        this->setId(++Integer::objN);
        this->setValue(integer.getValue());
    }
    ~Integer(){ // not required, having no pointer variable
    }
    Integer& operator=(const Integer &rhs) {
        this->setValue(rhs.getValue());
        return *this;
    }
    Integer operator+(const Integer &rhs){
        Integer res;
        res.setValue(this->getValue() * rhs.getValue());
        return res;
    }
}
```

Write setter function here for the static data member of class

```
};
long Integer::objN = 0;
```

```
int main(void){
    Integer i1(25);
    Integer i2(10);
    Integer a[2];
    a[0] = i1 + i2;
    a[1] = i2 + a[0];
    cout << "first sum is " << a[0].getValue() << endl;
    cout << " next sum is " << a[1].getValue() << endl;
    cout << endl;

    cout << "id of i1 " << i1.getId() << endl;
    cout << "id of i2 " << i2.getId() << endl;
    cout << "id of a[0] " << a[0].getId() << endl;
    cout << "id of a[1] " << a[1].getId() << endl;

    cout << endl;
    cout << "Next id is " << i1.objN+1 << endl;
    cout << "          or " << 1+Integer::objN << endl;
    return 0;
}
```

OUTPUT Should be given below